Expressões regulares

A mesma coisa, sem desenhar

Encontro 4 | 19 de agosto

Linguagens Formais e Compiladores

Três aulas desenhando

- Vocês leram autômatos, traçaram palavras e converteram para tabela.

Três aulas desenhando

- Vocês leram autômatos, traçaram palavras e converteram para tabela.
- Depois projetaram seis, do interruptor ao comentário de bloco.

Três aulas desenhando

- Vocês leram autômatos, traçaram palavras e converteram para tabela.
- Depois projetaram seis, do interruptor ao comentário de bloco.
- Funciona. E tem um problema prático.

Três aulas desenhando

- Vocês leram autômatos, traçaram palavras e converteram para tabela.
- Depois projetaram seis, do interruptor ao comentário de bloco.
- Funciona. E tem um problema prático.

Desenho não cabe num arquivo de configuração.

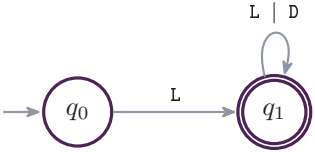
Três aulas desenhando

- Vocês leram autômatos, traçaram palavras e converteram para tabela.
- Depois projetaram seis, do interruptor ao comentário de bloco.
- Funciona. E tem um problema prático.

Desenho não cabe num arquivo de configuração.

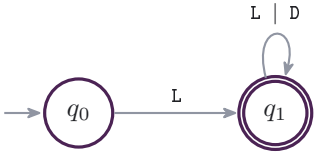
Hoje: uma notação de texto que diz exatamente a mesma coisa.

O identificador, nas duas notações




```
L (L | D)*
```

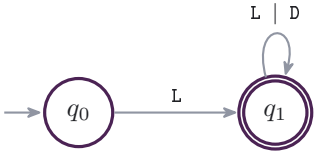
O identificador, nas duas notações




```
L (L | D)*
```

Mesma linguagem. Sete caracteres.

O identificador, nas duas notações




```
L (L | D)*
```

Mesma linguagem. Sete caracteres.

No fim da aula vocês vão saber ler e escrever a coluna da direita.

1
Três operadores

Só existem três

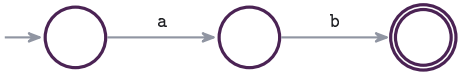
operador	**escreve-se**	**significa**	**exemplo**
concatenação	`ab`	primeiro `a`, depois `b`	`ab`
união	`a|b`	ou `a`, ou `b`	`a` ou `b`
fecho de Kleene	`a*`	zero ou mais `a`	ε, `a`, `aa`, ...

Só existem três

operador	**escreve-se**	**significa**	**exemplo**
concatenação	ab	primeiro a, depois b	ab
união	a|b	ou a, ou b	a ou b
fecho de Kleene	a*	zero ou mais a	ε, a, aa, ...

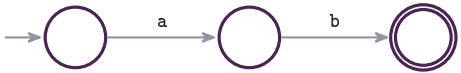
Tudo o mais que vocês vão ver é abreviação destes três.

Concatenação



ab

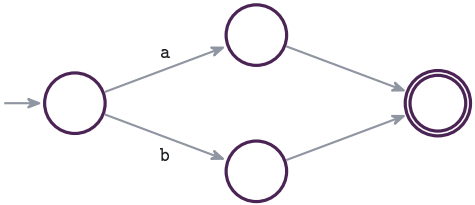
Concatenação



ab

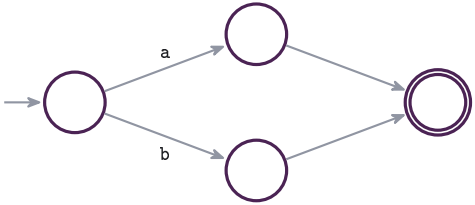
Um caminho em sequência. A ordem importa: ab não é ba.

União




```
a | b
```

União




```
a | b
```

Dois caminhos alternativos. Basta um deles servir.

Fecho de Kleene



a*

Fecho de Kleene



a*

Um laço. Repete quantas vezes quiser, **inclusive nenhuma**.

Fecho de Kleene



$a*$

Um laço. Repete quantas vezes quiser, **inclusive nenhuma**.

O ZERO É O QUE PEGA TODO MUNDO

$a*$ aceita a palavra vazia. Se você quer "pelo menos um", o operador é outro, e ele aparece daqui a três quadros.

2
Precedência

Sem parênteses, quem manda?

```
ab*
```

Sem parênteses, quem manda?

`ab*`

Isso é "a seguido de zero ou mais b",
ou "zero ou mais repetições de ab"?

Sem parênteses, quem manda?

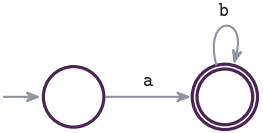
`ab*`

Isso é "a seguido de zero ou mais b",
ou "zero ou mais repetições de ab"?

A estrela gruda no símbolo imediatamente à esquerda.

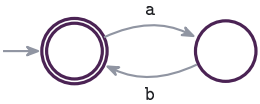
A diferença, desenhada

ab*



a, ab, abb, abbb

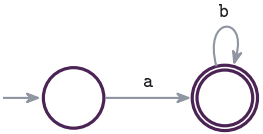
(ab)*



ε, ab, abab, ababab

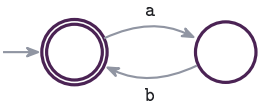
A diferença, desenhada

ab*



a, ab, abb, abbb

(ab)*



ε, ab, abab, ababab

Duas linguagens completamente diferentes. E a única diferença são dois parênteses.

A ordem completa

1º	fecho de Kleene	$*$	o mais forte
2º	concatenação	(nada)	
3º	união	$\mid$	o mais fraco

A ordem completa

1º	fecho de Kleene	`*`	o mais forte
2º	concatenação	(nada)	
3º	união	`|`	o mais fraco

`ab*`	é	a seguido de `(b*)`
`a|bc`	é	`a` ou `(bc)`
`ab|cd`	é	`(ab)` ou `(cd)`

A ordem completa

1º	fecho de Kleene	`*`	o mais forte
2º	concatenação	(nada)	
3º	união	`|`	o mais fraco

`ab*`	é	a seguido de `(b*)`
`a|bc`	é	`a` ou `(bc)`
`ab|cd`	é	`(ab)` ou `(cd)`

Na dúvida, use parênteses. Eles não custam nada.

3 Abreviações

Nenhuma delas é operador novo

escreve-se	é atalho para	significa
`a+`	`aa*`	um ou mais
`a?`	$\mathtt{a} \mid \varepsilon$	zero ou um
`[abc]`	$\mathtt{a} \mid \mathtt{b} \mid \mathtt{c}$	um destes
`[a-z]`	$\mathtt{a} \mid \mathtt{b} \mid \ldots \mid \mathtt{z}$	um da faixa
`.`	$s_1 \mid s_2 \mid \ldots$	qualquer símbolo

Nenhuma delas é operador novo

escreve-se	é atalho para	significa
a+	aa*	um ou mais
a?	a|ε	zero ou um
[abc]	a|b|c	um destes
[a-z]	a|b|...|z	um da faixa
.	s_1|s_2|...	qualquer símbolo

Toda expressão pode ser reescrita usando só concatenação, união e estrela.
As abreviações existem porque ninguém aguenta escrever a|b|c|...|z.

Uma promessa do encontro 2

autômato 1



a*

autômato 2



a+

Uma promessa do encontro 2

autômato 1



a*

autômato 2



a+

São os dois autômatos do encontro 2, que diferiam por um círculo duplo.

Uma promessa do encontro 2

autômato 1



a*

autômato 2



a+

São os dois autômatos do encontro 2, que diferiam por um círculo duplo.

Aquele círculo duplo é a estrela. A falta dele é o mais.

4

Por que as duas notações são equivalentes

A pergunta

Toda expressão regular tem um autômato equivalente?

E todo autômato tem uma expressão?

A pergunta

Toda expressão regular tem um autômato equivalente?

E todo autômato tem uma expressão?

Sim, nos dois sentidos.

A pergunta

Toda expressão regular tem um autômato equivalente?

E todo autômato tem uma expressão?

Sim, nos dois sentidos.

E o primeiro sentido não é só verdade: é uma **receita**.
Vocês vão ver a receita agora.

A ideia: montar por pedaços

Cada operador vira um pedacinho de autômato, com **uma entrada e uma saída**.



A ideia: montar por pedaços

Cada operador vira um pedacinho de autômato, com **uma entrada e uma saída**.



Como todo pedaço tem a mesma forma, dá para **encaixar** um no outro.

A ideia: montar por pedaços

Cada operador vira um pedacinho de autômato, com **uma entrada e uma saída**.



Como todo pedaço tem a mesma forma, dá para **encaixar** um no outro.

As transições sem rótulo são vazias: passam sem consumir símbolo.
Voltamos a elas no próximo encontro.

Os quatro pedaços

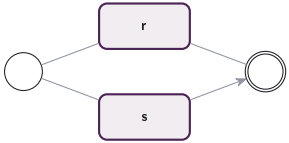
um símbolo `a`



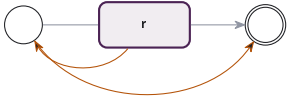
concatenação `rs`



união `r|s`



estrela `r*`



Os quatro pedaços

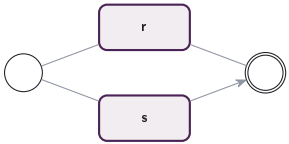
um símbolo `a`



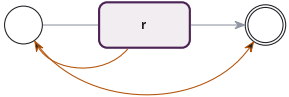
concatenação `rs`



união `r|s`



estrela `r*`



Na estrela, as duas flechas laranja: uma volta para repetir, outra pula tudo.

O que isso demonstra

Qualquer expressão pode ser montada encaixando esses quatro pedaços.

O que isso demonstra

Qualquer expressão pode ser montada encaixando esses quatro pedaços.

Logo, para toda expressão regular existe um autômato que reconhece a mesma linguagem.

O que isso demonstra

Qualquer expressão pode ser montada encaixando esses quatro pedaços.

Logo, para toda expressão regular existe um autômato que reconhece a mesma linguagem.

O NOME DISSO

Chama-se construção de Thompson, e é ela que está dentro de toda ferramenta que aceita expressão regular. Quando você escreve um padrão no editor, alguém montou esses pedaços por você.

5
Da oficina para o texto

Os autômatos da semana passada

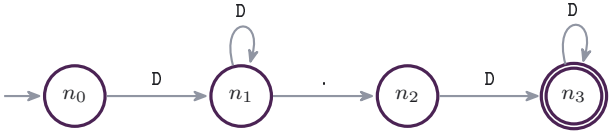
o que era	estados	a expressão
identificador	2	`[a-zA-Z_][a-zA-Z0-9_]*`
número inteiro	2	`[0-9]+`
número real	4	`[0-9]+.[0-9]+`
contém ab	3	`(a|b)*ab(a|b)*`

Os autômatos da semana passada

o que era	estados	a expressão
identificador	2	`[a-zA-Z_][a-zA-Z0-9_]*`
número inteiro	2	`[0-9]+`
número real	4	`[0-9]+.[0-9]+`
contém ab	3	`(a|b)*ab(a|b)*`

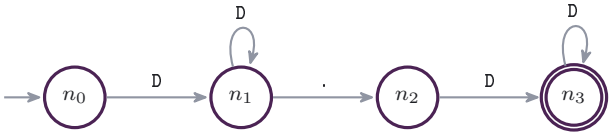
Quatro desenhos que levaram uma aula, em quatro linhas.

O número real, nas duas notações




```
[0-9]+\.[0-9]+
```

O número real, nas duas notações




```
[0-9]+\.[0-9]+
```

OBSERVE

Repare que o + nos dois lados do ponto é o que rejeita `12.` e `.5`. Se fosse `*`, os dois passariam. É o mesmo papel que o estado n_2 fazia ao não ser de aceitação.

E o comentário de bloco

```
/\*([^*]|\*+[^/])*\*+/
```

E o comentário de bloco

```
/\*([^*]|\*+[^/])*\*+/
```

Seis estados no desenho. Uma linha aqui, e uma linha bem feia.

E o comentário de bloco

```
/\*([^*]|\*+[^/])*\*+/
```

Seis estados no desenho. Uma linha aqui, e uma linha bem feia.

A LIÇÃO HONESTA

Expressão regular não é sempre melhor. Para padrões com estrutura interna, o desenho é mais legível. Para padrões simples, a expressão ganha de longe. Vocês vão usar as duas.

6
Padrões de token

Uma linguagem de verdade

token	padrão	exemplo
IDENT	[a-zA-Z_][a-zA-Z0-9_]*	total
INTEIRO	[0-9]+	42
REAL	[0-9]+\.[0-9]+	7.5
SE	se	se
ENQUANTO	enquanto	enquanto
ATRIB	=	=
IGUAL	==	==
BRANCO	[\t\n]+	

Uma linguagem de verdade

token	padrão	exemplo
IDENT	[a-zA-Z_][a-zA-Z0-9_]*	total
INTEIRO	[0-9]+	42
REAL	[0-9]+\.[0-9]+	7.5
SE	se	se
ENQUANTO	enquanto	enquanto
ATRIB	=	=
IGUAL	==	==
BRANCO	[\t\n]+	

É assim que se especifica a fase léxica de um compilador: uma tabela como esta.

Dois problemas aparecem imediatamente

se

Dois problemas aparecem imediatamente

se

Casa com `SE`. E casa com `IDENT`, porque começa com letra.
Dois padrões, a mesma entrada.

Dois problemas aparecem imediatamente

se

Casa com `SE`. E casa com `IDENT`, porque começa com letra.
Dois padrões, a mesma entrada.

==

Dois problemas aparecem imediatamente

se

Casa com `SE`. E casa com `IDENT`, porque começa com letra.
Dois padrões, a mesma entrada.

==

Casa com `IGUAL`. E o primeiro caractere sozinho casa com `ATRIB`.
Um padrão curto dentro de um longo.

As duas regras que resolvem

DEFINIÇÃO • CASAMENTO MAIS LONGO

Entre as alternativas, escolhe-se a que consome **mais caracteres**.

Resolve ==: duas letras vencem uma.

DEFINIÇÃO • PRIORIDADE POR ORDEM DE DECLARAÇÃO

Havendo empate no comprimento, vence o padrão **declarado primeiro**.

Resolve se: declare as palavras reservadas *antes* de `IDENT`.

As duas regras que resolvem

DEFINIÇÃO • CASAMENTO MAIS LONGO

Entre as alternativas, escolhe-se a que consome **mais caracteres**.

Resolve ==: duas letras vencem uma.

DEFINIÇÃO • PRIORIDADE POR ORDEM DE DECLARAÇÃO

Havendo empate no comprimento, vence o padrão **declarado primeiro**.

Resolve se: declare as palavras reservadas *antes* de `IDENT`.

As duas regras são convenção, e é a convenção que toda ferramenta adota.

O casamento mais longo, em ação



O casamento mais longo, em ação



O casamento mais longo, em ação



O casamento mais longo, em ação



O casamento mais longo, em ação



Em cada posição o analisador tenta todos os padrões e fica com o mais longo.
Depois recomeça da posição seguinte.

O casamento mais longo, em ação



Em cada posição o analisador tenta todos os padrões e fica com o mais longo.
Depois recomeça da posição seguinte.

O = isolado nunca é escolhido ali, porque == consome mais.

7

E1

A primeira etapa do projeto

Construir um analisador léxico a partir de autômatos.

forma	individual
entrega	encontro 8
o que se entrega	código funcionando, mais um relatório curto

A primeira etapa do projeto

Construir um analisador léxico a partir de autômatos.

forma	individual
entrega	encontro 8
o que se entrega	código funcionando, mais um relatório curto

É individual por decisão pedagógica: a passagem do autômato para o código é o exercício que consolida a unidade, e não deve ser terceirizada.

Como ela funciona

nível	o que passa a funcionar	o que você escreve
0	`qtd`	nada, já vem pronto
1	`42`	o autômato do inteiro
2	`qtd = 42;`	atribuição e ponto e vírgula
3	`inteiro qtd = 42;`	palavras reservadas
4	`real media = 7.5;`	o autômato do real
5	`logico ok = verdadeiro;`	mais palavras reservadas
6	`// comentário`	o autômato do comentário
7	um bloco completo	nada, é a recompensa

Como ela funciona

nível	o que passa a funcionar	o que você escreve
0	`qtd`	nada, já vem pronto
1	`42`	o autômato do inteiro
2	`qtd = 42;`	atribuição e ponto e vírgula
3	`inteiro qtd = 42;`	palavras reservadas
4	`real media = 7.5;`	o autômato do real
5	`logico ok = verdadeiro;`	mais palavras reservadas
6	`// comentário`	o autômato do comentário
7	um bloco completo	nada, é a recompensa

O testador para no primeiro nível que falha, e diz qual foi.

O que a oficina já preparou

- O autômato do número real, com a armadilha do `12.`, vocês já projetaram no papel.

O que a oficina já preparou

- O autômato do número real, com a armadilha do `12.`, vocês já projetaram no papel.
- O estado de erro absorvente vai ser necessário, e vocês já entenderam por quê.

O que a oficina já preparou

- O autômato do número real, com a armadilha do `12.`, vocês já projetaram no papel.
- O estado de erro absorvente vai ser necessário, e vocês já entenderam por quê.
- E a tabela de transições que vocês converteram é literalmente a estrutura de dados do código.

O que a oficina já preparou

- O autômato do número real, com a armadilha do `12.`, vocês já projetaram no papel.
- O estado de erro absorvente vai ser necessário, e vocês já entenderam por quê.
- E a tabela de transições que vocês converteram é literalmente a estrutura de dados do código.

Vocês não vão começar do zero. Vão codificar o que já desenharam.

O que fica desta aula

- Três operadores: concatenação, união e fecho. O resto é abreviação.

O que fica desta aula

- Três operadores: concatenação, união e fecho. O resto é abreviação.
- A estrela é mais forte que a concatenação, que é mais forte que a união.

O que fica desta aula

- Três operadores: concatenação, união e fecho. O resto é abreviação.
- A estrela é mais forte que a concatenação, que é mais forte que a união.
- $a*$ aceita vazio; $a+$ não. É o círculo duplo do encontro 2.

O que fica desta aula

- Três operadores: concatenação, união e fecho. O resto é abreviação.
- A estrela é mais forte que a concatenação, que é mais forte que a união.
- a∗ aceita vazio; a+ não. É o círculo duplo do encontro 2.
- Toda expressão vira autômato encaixando quatro pedaços.

O que fica desta aula

- Três operadores: concatenação, união e fecho. O resto é abreviação.
- A estrela é mais forte que a concatenação, que é mais forte que a união.
- a∗ aceita vazio; a+ não. É o círculo duplo do encontro 2.
- Toda expressão vira autômato encaixando quatro pedaços.
- Especificar a fase léxica é escrever uma tabela de padrões.

O que fica desta aula

- Três operadores: concatenação, união e fecho. O resto é abreviação.
- A estrela é mais forte que a concatenação, que é mais forte que a união.
- a∗ aceita vazio; a+ não. É o círculo duplo do encontro 2.
- Toda expressão vira autômato encaixando quatro pedaços.
- Especificar a fase léxica é escrever uma tabela de padrões.
- E dois padrões podem competir: resolve-se por comprimento, depois por ordem.

O que fica desta aula

- Três operadores: concatenação, união e fecho. O resto é abreviação.
- A estrela é mais forte que a concatenação, que é mais forte que a união.
- a* aceita vazio; a+ não. É o círculo duplo do encontro 2.
- Toda expressão vira autômato encaixando quatro pedaços.
- Especificar a fase léxica é escrever uma tabela de padrões.
- E dois padrões podem competir: resolve-se por comprimento, depois por ordem.

Próximo encontro: os pedaços de Thompson têm transições vazias.
Está na hora de falar de não-determinismo.

O desenho é para pensar.
A expressão é para escrever.

E as duas dizem exatamente a mesma coisa.